

COM668 Computing Project

Assessment Task 4 – Project Review Report

Project title: Privacy-First Personal Finance Assistant with Explainable Predictions and Chatbot Integration

Student name: Farhan Bin Hossain

Student ID number: B00970291

PSG identifier: PSG-Q2-09

Mentor name: Bakari Hardik

Course title: Computing Systems

Year of submission: 2025/2026

Word Count (2400 Maximum): 2044

Declaration:

I declare that this is all my own work. Any material I have referred to has been accurately referenced and any contribution of Artificial Intelligence technology has been fully acknowledged. I understand the importance of academic integrity and have read and understood the University's General Regulation: Student Academic Integrity and the Academic Misconduct Procedure. I understand that I must not upload my work before, during or after submission to any unapproved plagiarism detectors or answer sharing platforms, or equivalent, and that only University-approved platforms should be used.

Contents

1. Software realisation	1
1.1. Implementation review.....	1
1.2. Review of development tools used	1
2. Quality assurance.....	2
2.1. Description of verification and validation activities.....	2
2.2. Testing.....	2
3. Critical appraisal	3
3.1. Evaluation of project outcomes	3
3.2. Review of plans and any decisions	4
3.3. Evaluation of methodology	4
3.4. Personal development	5
References	5
Appendix A - Code Manifest	7
Additional appendices.....	8

1. Software realisation

1.1. Implementation review

This system follows the modular three layer architecture proposed in AT2 (Section 3.3.1), and every Must Have requirement was realised. The React 18 / Vite / TypeScript / Tailwind CSS frontend is deployed on Vercel and communicates over HTTPS with a Flask 3 backend on Render, backed by MongoDB Atlas. The principal components delivered are:

- **Security** — JWT authentication (PyJWT, bcrypt hashed credentials) combined with Auth0 OAuth extending the AT2 design with an enterprise grade sign in flow.
- **Encryption** — all personally identifiable fields, including user email, are encrypted at rest with AES-256-GCM and decrypted only within the authentication service, email is additionally HMAC-SHA256 hashed to allow indexed lookup without storing plaintext.
- **Forecasting and XAI** — a scikit learn regression model predicts monthly spending, evaluated with MAE, RMSE and R^2 as planned in AT2, with SHAP feature contributions rendered as human readable explanations on the forecast page.
- **Anomaly detection** — transactions exceeding the historical mean by more than two standard deviations are flagged with a plain-language alert, matching the AT2 statistical design.
- **Chatbot** — conversational expense logging via the Groq API (llama-3.1-8b-instant) as primary provider, with a local Ollama (llama3.1) fallback selectable through the LLM_PROVIDER environment variable an architectural evolution from AT2 discussed in Section 3.2.
- **Mobile** — a native iOS build via Capacitor, delivered beyond the AT2 scope, giving a working mobile MVP alongside the web application.
- **Research contribution** — fintech-llm-guard, a multi-layer LLM guardrail middleware package I designed and published to PyPI (v0.3.0), developed from the practical problem of securing the chatbot pipeline and now underpinning a GSAM 2026 paper submission in progress.

The most significant implementation challenge was Render's 512MB free-tier memory ceiling, which shaped the entire backend architecture: heavy libraries (NumPy, pandas) are lazy loaded inside functions rather than at module level, the LLM was moved off-process (Section 3.2), and fintech-llm-guard was decoupled from the production deployment. A second complex challenge a complete black screen failure on the iOS build was diagnosed to five compounding root causes and resolved (Section 2.2).

1.2. Review of development tools used

Tools were selected for being free tier accessible, well documented, and realistic for a solo developer shipping to production, rather than tools only viable for a funded team:

- **Development and CI/CD** — Visual Studio Code with separate React and Flask workspaces, Git/GitHub for version control, with Vercel and Render configured for automatic deployment from main a lightweight but genuine continuous-deployment pipeline.
- **MongoDB Atlas** — chosen over self-hosted MongoDB to remove database operations overhead, its free tier was sufficient for the project's data volumes while providing managed backups.

- **Postman and Newman** — a 38 endpoint collection across 8 functional folders, with Newman automating full collection runs and producing HTML reports stored in docs/ as durable test evidence.
- **Groq / Ollama** — Groq was selected over OpenAI for its generous free tier and low latency on small models, keeping the demonstration reliable, Ollama was retained for the fully local privacy mode.
- **Capacitor** — chosen over React Native because it wraps the existing, already tested web build rather than requiring a parallel codebase a decisive efficiency gain for a single developer.
- **Documentation** — architecture decisions were recorded as ADRs (e.g. docs/decisions/002-llm-hosting-deviation.md), and development history in a dated dev diary, giving traceable justification for every major choice.

2. Quality assurance

(suggested word count for Quality assurance: 800)

2.1. Description of verification and validation activities

Verification and validation ran continuously across every sprint rather than as a single end phase, matching the Agile Scrum methodology set out in AT2:

- **Verification** — each sprint increment was checked against its acceptance criteria before merging to main, with weekly mentor meetings acting as sprint reviews that surfaced design issues early the decision to encrypt PII fields was validated this way before rollout.
- **Input validation** — Pydantic v2 schemas validate every request at the API boundary, rejecting malformed input before it reaches the database layer, and TypeScript enforces contract consistency on the frontend.
- **Validation** — the system was validated against the user needs defined by the AT2 persona through structured usability testing (Section 2.2), and validation results directly changed the product not merely recorded.

2.2. Testing

Testing was organised into four formal levels, each with a defined method, tool and evidence artefact:

- **Model testing** — the forecasting model was evaluated with MAE, RMSE and R^2 against held out transaction history, per the AT2 evaluation plan, giving quantitative rather than anecdotal accuracy evidence.
- **API testing** — the Postman/Newman collection (38 endpoints) exercised expected and edge-case inputs invalid tokens, missing fields, unauthorised cross user access on every significant backend change, against both local and deployed environments. One representative defect caught this way: JWT_SECRET silently falling back to an insecure default caused all tokens to invalidate on restart; the fix requires the value to be explicitly set in .env.
- **Usability testing** — an in-app SUS (System Usability Scale) review sheet collected structured usability responses from test users, submitted via the Telegram Bot API for collation. Feedback directly refined the product: responsive behaviour was consolidated behind a single useDesktop() hook and modal widths standardised after participants reported inconsistent layouts between devices.

- **Cross-platform testing** — the Capacitor iOS build initially failed with a black screen. Systematic isolation traced this to five compounding causes: a `server.url` pointing at the Vercel deployment instead of the local bundle; a missing `base: './'` in the Vite config breaking asset paths; a corrupted native `ios/` scaffold; an absent production API URL; and a CORS allowlist not recognising `capacitor://`, `ionic://` and `localhost` origins. Each was fixed individually and login confirmed on device with a verified JWT response.
- **Security testing** — targeted checks aligned to the OWASP Top Ten: Flask Limiter's rate limiting on login/register was verified by scripted repeated requests until a 429 response; cross user isolation was tested by attempting to read another account's transactions with a valid but wrong user token (correctly rejected), and Pydantic's rejection of oversized, missing and wrongly typed fields was exercised through the Newman collection's negative case requests. Encryption at rest was verified by inspecting stored documents directly in Atlas and confirming PII fields hold only ciphertext.

Testing results were consistently used to improve the work, not only to record its state. The `JWT_SECRET` finding produced a hardened configuration policy, SUS feedback produced the `useIsDesktop()` consolidation and standardised modal widths; the iOS diagnosis produced a documented Capacitor deployment checklist so the fixes are repeatable; and every defect fix was followed by a full Newman regression run to confirm no endpoint behaviour had changed unintentionally. This verify–fix–reverify loop, run within each sprint, is the practical meaning of the Agile quality approach committed to in AT2.

Two defects remain open at the time of writing and are disclosed rather than hidden: the add transaction form's category selector still references a legacy unified category list, preventing income type entries from being categorised correctly, and a duplicate toast fires on transaction creation because both the modal and its parent `onAdded` callback call `showToast`. Both are logged and understood at code level. Their inclusion reflects an honest account of the verified state of the system.

3. Critical appraisal

(suggested word count for Critical appraisal: 1000)

3.1. Evaluation of project outcomes

Measured against the AT2 MoSCoW prioritisation, the project met its aims:

- **Must Have:** all delivered JWT/bcrypt authentication, manual and chatbot expense logging, regression forecasting, statistical anomaly detection, HTTPS communication, and SHAP based explainable outputs.
- **Should Have:** all delivered analytics dashboard, savings goal tracking, and monthly category summaries.
- **Could Have:** report export was not implemented; the what if simulation was consciously descoped in favour of the iOS build and the fintech-llm-guard research contribution both higher value uses of limited time than a feature the AT2 persona did not identify as a priority.

The AT2 performance and security objectives were also met: forecast generation completes within the two second target on the deployed Render instance, chatbot responses via Groq return with latency suitable for interactive use (a direct benefit of ADR-002), all endpoints are HTTPS only, and no raw financial records are transmitted to any third-party AI service. Validation results support this assessment as a whole: Newman

reports evidence correct endpoint behaviour across environments, model metrics quantify forecast accuracy, and SUS responses from test users confirm the interface is usable by the non-technical audience the persona represents. The most significant outcome beyond the AT2 scope is fintech-llm-guard a published, versioned PyPI package that turned an internal security necessity into a reusable, citable artefact now forming the empirical basis of a GSAM 2026 submission.

3.2. Review of plans and any decisions

The AT2 plan organised the project into two-week sprints across 20 weeks. The plan was actively managed rather than followed rigidly: the sprint structure held for the core build (authentication through XAI outputs), but the later UI Polish and System Review sprints were partially re-purposed to absorb two unplanned workstreams the Capacitor iOS build and the extraction of fintech-llm-guard with the dissertation writeup buffer protecting the AT3 and AT4 deadlines. Progress was tracked against the Gantt baseline and replanned at the weekly mentor meetings when deviations emerged, which is how both major changes below were absorbed without missing any assessed deadline.

The most consequential decision was the chatbot hosting model. AT2 specified a fully local Ollama architecture; production uses Groq's cloud API as primary, with Ollama retained as a selectable local fallback (ADR-002). This was not an abandonment of the privacy thesis but a reframing into a four-tier hybrid model: users can disable the cloud chatbot entirely, restoring fully local operation, while the default trades a narrow, well-scoped exposure chat messages only, never raw transaction record for the latency and reliability a live demonstration requires. The trigger was Render's 512MB ceiling, within which a local LLM runtime alongside scikit-learn, pandas and NumPy was not viable.

The same constraint drove the second decision: fintech-llm-guard was removed from the production requirements.txt after pushing the backend past the memory limit despite working correctly in isolation. I chose a stable deployment plus a cleanly separated, independently published research artefact over a monolithic but memory fragile deployment, with microservice decomposition documented as the path back to integration. Given a fixed, self funded infrastructure budget, I judge both decisions correct, and both are fully documented in ADRs so the reasoning is auditable.

Reviewing the AT2 risk analysis against what actually occurred: the 'performance bottleneck' risk materialised in a form the analysis did not anticipate as a hosting memory ceiling rather than model inference time and its mitigation (lazy loading, provider abstraction) went beyond what was planned. The 'unauthorised access' and 'sensitive data exposure' risks were mitigated more strongly than proposed, adding field-level AES-256-GCM encryption and rate limiting to the planned bcrypt/JWT/HTTPS baseline. The 'forecasting inaccuracy' risk was managed exactly as planned through MAE/RMSE validation. One risk the AT2 register missed entirely was platform deployment complexity, which consumed a full sprint on the iOS build; a future register would include a deployment-target risk with an explicit time-box.

3.3. Evaluation of methodology

Agile Scrum, selected in AT2 over Waterfall and Iterative models, proved the right choice for a project of semi independent components whose feasibility could not be fully known in advance. Both hosting deviations emerged from constraints discovered during implementation, not requirements analysis, and

scrum's sprint review loop in practice, weekly mentor meetings let each be absorbed and replanned within a sprint. A Waterfall approach with deferred testing would likely have surfaced the memory ceiling only after both Ollama and fintech-llm-guard were fully integrated, at far higher cost. The one honest weakness: as a solo developer holding all three Scrum roles, sprint discipline occasionally slipped when deep in debugging (notably the iOS week), confirming the AT2 risk assessment that Scrum 'requires discipline'; the dated dev diary was the mechanism that pulled planning back on track.

3.4. Personal development

This project taught me to engineer under real constraints rather than idealised ones. Debugging the iOS black screen taught systematic isolation of compounding causes over symptom-patching. The Render ceiling forced genuine architectural trade offs, and recording them as ADRs is a professional habit I will keep. Publishing fintech-llm-guard was my first experience of open-source package maintenance and venue-directed academic writing, and it has redirected my career intent toward AI safety and secure system design alongside full stack engineering. Most importantly, I learned that deviating from a plan transparently, with documented reasoning, is a strength of an engineering process not a failure of it.

References

(Section not included in word count. Must be in the Harvard style)

Arrieta, A.B., Díaz-Rodríguez, N., Del Ser, J., Benetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R. and Chatila, R. (2020). 'Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI', *Information Fusion*, 58, pp. 82–115.

Brooke, J. (1996). 'SUS: A “quick and dirty” usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189–194.

Dworkin, M.J. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. Gaithersburg: National Institute of Standards and Technology.

European Parliament (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation). [online] Official Journal of the European Union. Available at: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679> [Accessed July 2026].

Hyndman, R. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. 3rd ed. [online] OTexts. Available at: <https://otexts.com/fpp3> [Accessed July 2026].

Information Commissioner's Office (ICO) (2023). *Guide to the UK GDPR*. [online] ICO. Available at: <https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr> [Accessed July 2026].

Lundberg, S.M. and Lee, S.I. (2017). 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, 30, pp. 4765–4774.

Molnar, C. (2019). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. [online] Available at: <https://christophm.github.io/interpretable-ml-book> [Accessed July 2026].

OWASP Foundation (2021). *OWASP Top Ten*. [online] Available at: <https://owasp.org/www-project-top-ten/> [Accessed July 2026].

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R. and Dubourg, V. (2011). 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Provos, N. and Mazières, D. (1999). 'A future-adaptable password scheme', *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*, pp. 81–91.

Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum*. [online] Available at: <https://scrumguides.org> [Accessed July 2026].

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P. and Bhosale, S. (2023). *Llama 2: Open foundation and fine-tuned chat models*. [online] arXiv.org. Available at: <https://arxiv.org/abs/2307.09288> [Accessed July 2026].

Appendix A - Code Manifest

(Section not included in word count)

Filename	Contribution	Purpose	Path to file (in AT3 zip file)
app/__init__.py	Own work. Claude assisted with CORS allowlist origins.	Flask application factory; registers blueprints, CORS allowlist (incl. Capacitor origins), encryption and rate-limiter initialisation	backend/app/__init__.py
auth_service.py	Own work. Claude assisted with debugging	Registration/login logic; verifies bcrypt hash, decrypts stored email field, issues JWT access/refresh pair	backend/app/services/auth_service.py
encryption.py	Own work. Claude assisted.	AES-256-GCM field-level encryption/decryption helpers for PII at rest; HMAC-SHA256 email hashing for indexed lookup without storing plaintext	backend/app/core/encryption.py
groq_client.py	Own work. Claude assisted with the LLMClientProtocol	GroqLLMClient wrapper; selects Groq or Ollama provider via LLM_PROVIDER environment variable	backend/app/services/groq_client.py
transaction_service.py	Own work. Claude assisted.	Transaction CRUD; get_all_transactions_for_analysis() feeds the forecasting and anomaly pipeline, excludes soft-deleted records	backend/app/services/transaction_service.py
forecast_service.py	Own work. Claude assisted with SHAP integration.	scikit-learn regression forecasting model; SHAP feature-contribution explainability layer; MAE/RMSE/R ² evaluation	backend/app/services/forecast_service.py
limiter.py	Own work. Claude assisted with module level singleton pattern.	Flask-Limiter singleton applied to authentication endpoints; prevents brute-force attacks	backend/app/core/limiter.py
App.tsx / pages/	Own work. Claude assisted with responsive layout debugging .	React SPA shell, routing, and page components (Dashboard, Transactions, Chat, Goals, Forecast)	frontend/src/App.tsx
ReviewSheet.tsx	Own work. Concept, SUS question set, and Telegram delivery integration designed and implemented by student.	In-app SUS usability questionnaire; submits structured participant responses via Telegram Bot API	frontend/src/components/ReviewSheet.tsx
vite.config.ts	Own work. Claude identified and assisted debug.	Vite build config; base: './' required for Capacitor-compatible relative asset paths on iOS	frontend/vite.config.ts
capacitor.config.ts / ios/	Own work. Claude assisted with systematic diagnosis	Capacitor iOS wrapper; regenerated native scaffold serving local bundle, with verified on-device JWT login	frontend/capacitor.config.ts
fintech_llm_guard (PyPI v0.3.0)	Student conceived research contribution.	Multi-layer LLM guardrail middleware; published independent package.	https://pypi.org/project/fintech-llm-guard/

Additional appendices

(Section not included in suggested word count)

Note, appendices must not include narrative, discussion, analysis, or similar writing. They must only include additional items that evidence practice claimed, discussed or analysed within the main body of the report.